

Agentic RAG Systems for Automated Accessibility Compliance: A Production Case Study at Evenness

Kaustubha Venkata Eluri

Agentic AI Engineer – RAG Systems & Production Microservices

Evenness, Inc.

Atlanta, GA, USA

Abstract—This paper documents production engineering work on agentic, retrieval-augmented-generation (RAG) systems for automated digital-accessibility compliance analysis, performed as an Agentic AI Engineer at Evenness, Inc. The work covered building a RAG-backed vector knowledge base spanning WCAG 2.0/2.1/2.2, ADA, AODA, and EEA compliance standards with versioned compliance selection; integrating agent workflows into four production microservices; and automating multi-step audit pipelines. We report the reliability, throughput, and time-savings outcomes documented from this engagement and discuss the systems-engineering practices – reliability targets on agentic microservices, pipeline automation, and observability – that produced them.

Index Terms—retrieval-augmented generation, agentic AI, microservices, accessibility compliance, WCAG, production ML systems, observability

I. INTRODUCTION

Digital-accessibility compliance analysis – determining whether a web application or digital product satisfies standards such as WCAG, ADA, AODA, and the EU’s European Accessibility Act (EEA) – is a domain that combines regulatory precision with a large and constantly evolving rule corpus across jurisdictions and standard versions. Manually auditing an application against this corpus does not scale to the frequency at which production applications change. This motivates an agentic, retrieval-augmented approach: an AI agent workflow that retrieves the specific, version-correct compliance rules relevant to a given piece of content or code, then applies them, rather than relying on a static, hardcoded rule set or an unconstrained language model with no grounding in the actual standard text.

II. ROLE AND SCOPE

As an Agentic AI Engineer at Evenness, the author’s scope covered designing and deploying RAG-backed AI workflows across production microservices to automate accessibility compliance analysis, spanning both the retrieval/knowledge-base layer and the production infrastructure that served it.

III. RAG-BACKED VECTOR KNOWLEDGE BASE WITH VERSIONED COMPLIANCE SELECTION

The core system built during this engagement was a RAG-backed vector knowledge base covering WCAG 2.0, WCAG 2.1, WCAG 2.2, ADA, AODA, and EEA standards, with versioned compliance selection – the ability to retrieve and

apply the specific version of a standard relevant to a given audit context, rather than treating all versions of a standard as interchangeable. This is a meaningful distinction in accessibility compliance: WCAG 2.1 and WCAG 2.2 share most but not all success criteria, and a compliance analysis that conflates versions can produce both false positives (flagging a criterion that does not apply under the selected version) and false negatives (missing a newer criterion). Achieving 100% WCAG 2.1 rule coverage required the knowledge base to represent this rule corpus with sufficient fidelity that unified evaluation and reporting workflows could operate against it without gaps.

IV. AGENTIC WORKFLOWS IN PRODUCTION MICROSERVICES

Agent workflows built on top of this knowledge base were integrated into four production microservices, sustaining 99.9% uptime and a p95 latency below 300ms. Sustaining sub-300ms p95 latency for an agentic workflow – which typically involves at least one retrieval step and one generation step, and often several tool calls – requires careful attention to retrieval efficiency and prompt/context size, since naive agentic architectures with unconstrained retrieval or excessive tool-call chaining commonly exceed this latency bar by an order of magnitude. The 99.9% uptime figure across four production microservices further indicates that the agentic components were engineered with the same reliability discipline as conventional, non-AI production services, rather than being treated as an experimental layer with looser availability expectations.

V. AUDIT PIPELINE AUTOMATION

Beyond the underlying RAG and agent infrastructure, the engagement automated multi-step audit pipelines – the end-to-end process of taking a piece of content or code, retrieving the relevant compliance rules, applying them, and producing a structured compliance report. This automation reduced review cycle time from 15 minutes to 9 minutes per audit and saved over 120 engineering hours per quarter in aggregate. A 40% reduction in per-audit review time, multiplied across a production audit volume, is the primary mechanism by which an agentic automation system converts model-level correctness into organizational time savings; the 120-hours-per-quarter

figure represents that aggregate effect at the volume of audits actually run in production during the engagement.

VI. OBSERVABILITY AND INCIDENT RESPONSE

Running agentic workflows in production introduces a monitoring surface that does not exist in conventional deterministic services: agent execution paths, tool-call outcomes, and retrieval quality all need to be observable in order to diagnose degraded behavior. The author built monitoring and observability tooling to track agent execution and system health, reducing mean time to resolution (MTTR) by 30%. This reflects a general principle in operating agentic systems in production: because agent behavior is less predictable step-to-step than a conventional deterministic service, the return on investment for observability tooling – execution tracing, tool-call logging, retrieval-quality metrics – is higher than in a conventional service, precisely because it is the primary tool available for diagnosing a failure that cannot simply be reproduced by re-reading a stack trace.

VII. DISCUSSION

The work at Evenness illustrates a pattern in taking agentic AI systems from prototype to production: the AI/retrieval component (the versioned compliance knowledge base) and the systems-engineering component (uptime, latency, observability, pipeline automation) are both necessary, and the systems-engineering work is what converts a working retrieval system into something that can sustain 99.9% uptime across four production microservices and save 120 engineering hours per quarter. The versioned compliance selection design in particular reflects a domain-specific correctness requirement – treating standard versions as distinct rather than interchangeable – that a generic RAG system would not automatically satisfy without deliberate schema and retrieval design.

VIII. LIMITATIONS

The metrics reported in this paper (uptime, latency, review-cycle-time reduction, MTTR reduction) are drawn from the author’s own professional record of the engagement and reflect internal production measurements at Evenness rather than a published, independently reproducible benchmark. Details of the specific vector database, embedding model, or agent orchestration framework used are not part of the author’s documented record and are therefore not claimed here.

IX. CONCLUSION

This case study documents the engineering required to operate an agentic, retrieval-augmented accessibility-compliance system in production: a versioned compliance knowledge base spanning five standards, agent workflows sustaining 99.9% uptime and sub-300ms p95 latency across four microservices, automated audit pipelines that cut review time by 40% and saved over 120 engineering hours per quarter, and observability tooling that reduced MTTR by 30%. These outcomes reflect the combined necessity of retrieval correctness and systems-engineering discipline in operating agentic AI at production reliability standards.